

Primary sources & further reading

Track 16.4 —  LangGraph · 16-ai-tech-stack/tracks/langgraph/papers/PAPERS.md · 16-ai-tech-stack/tracks/langgraph/resources/RESOURCES.md

Primary Sources — LangGraph

LangGraph is a library, not a research programme, so the "papers" here split three ways: the **official docs** (the only authority on the current API), the **research the design implements**, and the **field reports** that argue about how far to take it. Read the docs first; they change, and a blog post from last year will teach you a deprecated API.

All links verified September 2026. If a page has moved, start from <https://docs.langchain.com/oss/python/langgraph/overview>.

Read these first (the actual API)

#	Source	Why it matters	Link
1	Graph API	The definitive reference for <code>StateGraph</code> , nodes, edges, reducers, <code>Send</code> , <code>Command</code> , subgraphs, caching and retries. Everything in <code>notes/01</code> is a compression of this.	https://docs.langchain.com/oss/python/langgraph/graph-api
2	Persistence	Checkpointers, threads, <code>StateSnapshot</code> , time travel, stores. The page that explains what you are actually buying.	https://docs.langchain.com/oss/python/langgraph/persistence
3	Human-in-the-loop / interrupts	<code>interrupt()</code> , <code>Command(resume=...)</code> , and the re-execution semantics that cause the classic double-side-effect bug.	https://docs.langchain.com/oss/python/langgraph/interrupts
4	Streaming	All stream modes, <code>get_stream_writer()</code> , <code>subgraphs=True</code> . Read before you design a UI, not after.	https://docs.langchain.com/oss/python/langgraph/streaming
5	Durable execution	Durability modes, failure recovery, why nodes must be replay-safe.	https://docs.langchain.com/oss/python/langgraph/durable-execution

Also worth bookmarking:

- **Subgraphs** — <https://docs.langchain.com/oss/python/langgraph/use-subgraphs>
- **Workflows and agents** (the pattern catalogue: prompt chaining, routing, orchestrator-worker, evaluator-optimizer) — <https://docs.langchain.com/oss/python/langgraph/workflows-agents>
- **Functional API** (`@entrypoint` / `@task` — the same runtime without drawing a graph) — <https://docs.langchain.com/oss/python/langgraph/use-functional-api>

- **What's new in LangGraph v1** (including `create_react_agent` → `create_agent`) — <https://docs.langchain.com/oss/python/releases/langgraph-v1>
- **API reference** — <https://reference.langchain.com/python/langgraph/>
- **Source** — <https://github.com/langchain-ai/langgraph>

The research LangGraph implements

Paper	Year	One-line takeaway	Link
Pregel: A System for Large-Scale Graph Processing — Malewicz et al.	2010	The bulk-synchronous superstep model LangGraph's executor is named after and built on. Read §2–3 and the whole "why parallel branches join" question dissolves.	Google Research · PDF
ReAct: Synergizing Reasoning and Acting in Language Models — Yao et al.	2022	The agent/tools cycle in <code>code/react_agent_from_scratch.py</code> , in its original form.	arXiv:2210.03629
Reflexion: Language Agents with Verbal Reinforcement Learning — Shinn et al.	2023	Self-critique as a <i>loop with memory</i> — i.e. the draft/critique cycle, and why the critique must land in state.	arXiv:2303.11366
Tree of Thoughts — Yao et al.	2023	Branching and backtracking over a search tree. Time travel + forking is the operational version of this.	arXiv:2305.10601
AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation — Wu et al.	2023	The conversation-centric alternative to graph-centric orchestration. Useful contrast: what you give up for flexibility.	arXiv:2308.08155
MetaGPT: Meta Programming for Multi-Agent Collaborative Framework	2023	Role-based hierarchical teams, and the coordination overhead that comes with them.	arXiv:2308.00352
Cognitive Architectures for Language Agents (CoALA) — Sumers et al.	2023	A framework for talking about agent memory, action space and decision procedure. Good vocabulary for justifying a state schema.	arXiv:2309.02427
Executable Code Actions Elicit Better LLM Agents (CodeAct) — Wang et al.	2024	Sometimes the right "tool node" is a sandboxed interpreter, not twelve JSON schemas.	arXiv:2402.01030

Field reports — read both sides

Source	Position	Link
Building Effective Agents — Anthropic	The canonical "workflows vs. agents" distinction, and an argument for the simplest thing that works. Read it before your first graph and again after your third.	https://www.anthropic.com/engineering/building-effective-agents
How we built our multi-agent research system — Anthropic	The pro-multi-agent case, with numbers: their multi-agent system used roughly 15× the tokens of a plain chat. Worth it for parallel research; absurd for a support bot.	https://www.anthropic.com/engineering/multi-agent-research-system
Don't Build Multi-Agents — Cognition	The anti-multi-agent case: context fragmentation and lossy hand-offs make distributed agents fragile. The strongest argument for "one agent, better tools".	https://cognition.ai/blog/dont-build-multi-agents
How to think about agent frameworks — LangChain	The vendor's own framing of where orchestration frameworks help and where they get in the way. Read critically; it is still the clearest map.	https://blog.langchain.com/how-to-think-about-agent-frameworks/
LangChain and LangGraph reach v1.0	What stabilised in October 2025 and what the 1.x contract is.	https://www.langchain.com/blog/langchain-langgraph-1dot0

How to read this material (45 minutes, 3 passes)

1. **Pass 1 (10 min)**: read the docs page for the *one* feature you need. Run its code snippet. Docs beat blogs here — the API moved between 0.2, 0.4 and 1.0, and the internet did not notice.
2. **Pass 2 (20 min)**: open the corresponding LangGraph source file. It is small and readable. `libs/checkpoint/` in particular is a short, honest piece of engineering.
3. **Pass 3 (15 min)**: find the failure mode. For every feature ask: what happens on retry, on replay, on two concurrent writers, and on a 3-hour pause? Write the answer down. That note is worth more in six months than the page you read it from.

Resources — LangGraph

Documentation (keep two tabs open)

Page	What's inside
https://langchain-ai.github.io/langgraph/	The canonical LangGraph docs: graphs, state, checkpointing, interrupts
https://langchain-ai.github.io/langgraph/concepts/	The concept guides — read before the how-tos; the mental model lives here
https://langchain-ai.github.io/langgraph/how-tos/	Task-oriented recipes: retries, subgraphs, time travel, map-reduce
https://docs.langchain.com/	The unified LangChain/LangGraph docs home (v1 era)

Repositories

Repo	What's inside
https://github.com/langchain-ai/langgraph	The library itself; <code>libs/langgraph</code> is readable and heavily commented
https://github.com/langchain-ai/langgraphjs	The TypeScript twin — worth a skim for the shared mental model
https://github.com/langchain-ai/langgraph-examples	Runnable example apps (agents, RAG, customer support)

Courses (free)

Course	Why	Link
LangChain Academy — Intro to LangGraph	The official course, taught by the framework authors; state and human-in-the-loop chapters are the good parts	https://academy.langchain.com/courses/intro-to-langgraph
DeepLearning.AI — AI Agents in LangGraph	Short, hands-on, agentic search and multi-agent examples	https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/

Blogs & papers worth your time

- The LangGraph concept guide on persistence is the best short explanation of checkpointing semantics — read it after `../code/checkpoint_and_interrupt.py`, not before.
- **ReAct** ([arXiv:2210.03629](https://arxiv.org/abs/2210.03629)) — the loop you rebuild by hand in this track
- **Building Effective Agents** — Anthropic: <https://www.anthropic.com/engineering/building-effective-agents> (the workflows-vs-agents distinction that decides whether you need this library at all)
- The annotated paper list for this track: `../papers/PAPERS.md`

Ecosystem neighbours

Tool	Relationship to LangGraph
https://smith.langchain.com/	Tracing + evals; the observability half of the story (<code>../langsmith/</code>)
https://www.temporal.io/	The durable-execution engine to compare against when reliability is the driver
https://github.com/crewAIInc/crewAI	CrewAI: the role-based multi-agent alternative — less explicit, less controllable
https://github.com/microsoft/autogen	AutoGen: conversational multi-agent; the other main orchestration camp

Communities

- LangChain forum — <https://forum.langchain.com/>
- GitHub Discussions on the langgraph repo — the maintainers answer; search before filing